

CO2- LAB EXPERIMENTS

1. Python Program – Map Coloring using CSP

Code

```
colors = ["Red", "Green", "Blue"]
```

```
neighbors = {  
    "A": ["B", "C"],  
    "B": ["A", "C", "D"],  
    "C": ["A", "B", "D"],  
    "D": ["B", "C"]  
}
```

```
def is_valid(region, color, assignment):  
    for neighbor in neighbors[region]:  
        if neighbor in assignment and assignment[neighbor] == color:  
            return False  
    return True
```

```
def solve(assignment):  
    if len(assignment) == len(neighbors):  
        return assignment
```

```
    region = next(r for r in neighbors if r not in assignment)
```

```
    for color in colors:  
        if is_valid(region, color, assignment):  
            assignment[region] = color
```

```
result = solve(assignment)
```

```
if result:
```

```
    return result
```

```
del assignment[region]
```

```
return None
```

```
solution = solve({})
```

```
print("Map Coloring Solution:")
```

```
for region, color in solution.items():
```

```
    print(region, ":", color)
```

Input

No input required.

Output

Map Coloring Solution:

A : Red

B : Green

C : Blue

D : Red

The screenshot shows a code editor with a file named 'main.py'. The code implements a recursive map coloring algorithm. It defines a 'solve' function that takes an 'assignment' dictionary. If the assignment is complete (all neighbors are colored), it returns the assignment. Otherwise, it finds an uncolored region, tries each color, and recursively solves the remaining regions. The main code calls 'solve({})' and prints the solution.

```
main.py Visualize Run
```

```
16 def solve(assignment):
17     if len(assignment) == len(neighbors):
18         return assignment
19
20     region = next(r for r in neighbors if r not in assignment)
21
22     for color in colors:
23         if is_valid(region, color, assignment):
24             assignment[region] = color
25
26             result = solve(assignment)
27             if result:
28                 return result
29
30     del assignment[region]
31
32     return None
33
34 solution = solve({})
35 print("Map Coloring Solution:")
36 for region, color in solution.items():
37     print(region, ":", color)
38
```

Output

```
Map Coloring Solution:
A : Red
B : Green
C : Blue
D : Red

=== Code Execution Successful ===
```

2. Python Program – Tic Tac Toe

Code

```
board = [" "] * 9
```

```
def display():
```

```
    print(board[0], "|", board[1], "|", board[2])
```

```
    print("--+---+--")
```

```
    print(board[3], "|", board[4], "|", board[5])
```

```
    print("--+---+--")
```

```
    print(board[6], "|", board[7], "|", board[8])
```

```
def check_winner(player):
```

```
    wins = [
```

```
        (0,1,2), (3,4,5), (6,7,8),
```

```
        (0,3,6), (1,4,7), (2,5,8),
```

```
        (0,4,8), (2,4,6)
```

```
    ]
```

```
    return any(all(board[i] == player for i in w) for w in wins)
```

```
for turn in range(9):
```

```
    display()
```

```
    player = "X" if turn % 2 == 0 else "O"
```

```
    position = int(input(player + " enter position (1-9): ")) - 1
```

```
    if board[position] != " ":
```

```
        print("Position already occupied!")
```

```
        continue
```

```
board[position] = player
```

```
if check_winner(player):
```

```
    display()
```

```
    print(player, "wins!")
```

```
    break
```

```
else:
```

```
    display()
```

```
    print("It's a draw!")
```

Input

X enter position (1-9): 1

O enter position (1-9): 2

X enter position (1-9): 5

O enter position (1-9): 3

X enter position (1-9): 9

O enter position (1-9): 4

X enter position (1-9): 7

Output

X wins!

```
Output
  |  |
  ---
  |  |
  ---
  |  |
X enter position (1-9): 6
  |  |
  ---
  |  | X
  ---
  |  |
O enter position (1-9): 6
Position already occupied!
  |  |
  ---
  |  | X
  ---
  |  |
X enter position (1-9): 6
Position already occupied!
  |  |
  ---
  |  | X
  ---
  |  |
O enter position (1-9): 7
  |  |
  ---
  |  | X
  ---
O |  |
X enter position (1-9): 3
  |  | X
  ---
  |  | X
  ---
O |  |
O enter position (1-9): 5
  |  | X
  ---
  | O | X
  ---
O |  |
X enter position (1-9): 2
  | X | X
  ---
  | O | X
  ---
O |  |
O enter position (1-9): 4
  | X | X
  ---
O | O | X
  ---
O |  |
X enter position (1-9): 6
Position already occupied!
  | X | X
  ---
O | O | X
  ---
O |  |
It's a draw!

=== Code Execution Successful ===
```

3. Python Program – Minimax Algorithm for Gaming

Code

```
def minimax(depth, nodeIndex, maximizingPlayer, values, height):

    if depth == height:

        return values[nodeIndex]
```

```
if maximizingPlayer:
    return max(
        minimax(depth + 1, nodeIndex * 2, False, values, height),
        minimax(depth + 1, nodeIndex * 2 + 1, False, values, height)
    )
else:
    return min(
        minimax(depth + 1, nodeIndex * 2, True, values, height),
        minimax(depth + 1, nodeIndex * 2 + 1, True, values, height)
    )
```

```
values = [3, 5, 2, 9, 12, 5, 23, 23]
```

```
height = 3
```

```
result = minimax(0, 0, True, values, height)
```

```
print("Leaf Values:", values)
```

```
print("Best Value using Minimax:", result)
```

Input

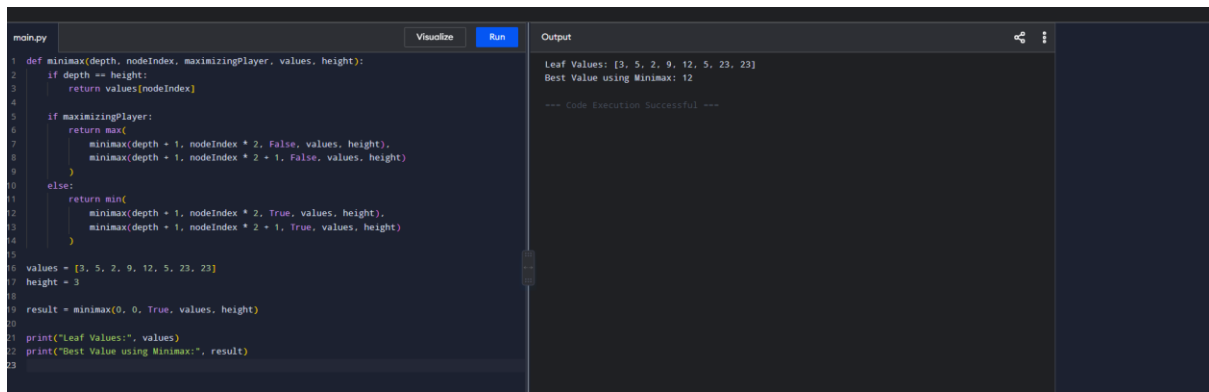
Leaf Values:

3 5 2 9 12 5 23 23

Output

Leaf Values: [3, 5, 2, 9, 12, 5, 23, 23]

Best Value using Minimax: 12



The screenshot shows a Python IDE with a file named 'main.py'. The code implements a Minimax algorithm for a game tree. The 'minimax' function takes depth, nodeIndex, maximizingPlayer, values, and height as parameters. It returns the value of the node at the given depth. The 'values' list is [3, 5, 2, 9, 12, 5, 23, 23]. The 'height' is 3. The 'result' is calculated as minimax(0, 0, True, values, height). The output shows 'Leaf Values: [3, 5, 2, 9, 12, 5, 23, 23]' and 'Best Value using Minimax: 12'. The code execution is successful.

```
1 def minimax(depth, nodeIndex, maximizingPlayer, values, height):
2     if depth == height:
3         return values[nodeIndex]
4
5     if maximizingPlayer:
6         return max(
7             minimax(depth + 1, nodeIndex * 2, False, values, height),
8             minimax(depth + 1, nodeIndex * 2 + 1, False, values, height)
9         )
10    else:
11        return min(
12            minimax(depth + 1, nodeIndex * 2, True, values, height),
13            minimax(depth + 1, nodeIndex * 2 + 1, True, values, height)
14        )
15
16 values = [3, 5, 2, 9, 12, 5, 23, 23]
17 height = 3
18
19 result = minimax(0, 0, True, values, height)
20
21 print("Leaf Values:", values)
22 print("Best Value using Minimax:", result)
23
```

Output:

```
Leaf Values: [3, 5, 2, 9, 12, 5, 23, 23]
Best Value using Minimax: 12
--- Code Execution Successful ---
```

4. Python Program – Alpha-Beta Pruning

Code

```
def alphabeta(depth, nodeIndex, maximizingPlayer, values, alpha, beta, height):
```

```
    if depth == height:
```

```
        return values[nodeIndex]
```

```
    if maximizingPlayer:
```

```
        best = -9999
```

```
        for i in range(2):
```

```
            value = alphabeta(
                depth + 1, nodeIndex * 2 + i,
                False, values, alpha, beta, height
            )
```

```
            best = max(best, value)
```

```
            alpha = max(alpha, best)
```

```
            if beta <= alpha:
```

```
                break
```

```
    return best
```

```

else:
    best = 9999

    for i in range(2):
        value = alphabeta(
            depth + 1, nodeIndex * 2 + i,
            True, values, alpha, beta, height
        )
        best = min(best, value)
        beta = min(beta, best)

    if beta <= alpha:
        break

    return best

values = [3, 5, 2, 9, 12, 5, 23, 23]
height = 3

result = alphabeta(0, 0, True, values, -9999, 9999, height)

print("Leaf Values:", values)
print("Best Value using Alpha-Beta Pruning:", result)

```

Input

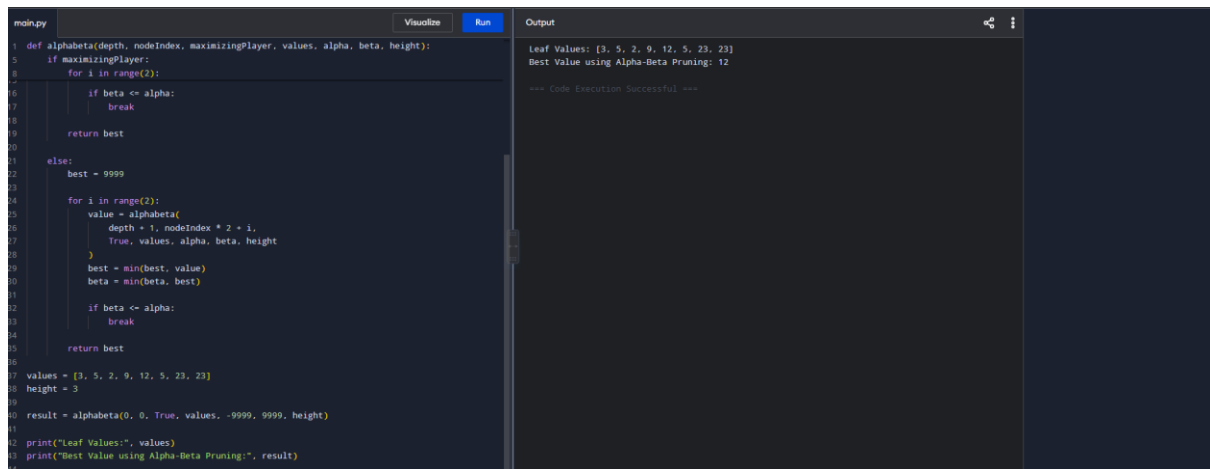
Leaf Values:

3 5 2 9 12 5 23 23

Output

Leaf Values: [3, 5, 2, 9, 12, 5, 23, 23]

Best Value using Alpha-Beta Pruning: 12



The screenshot shows a Jupyter Notebook interface with a code editor on the left and an output area on the right. The code defines an `alphabeta` function that implements Alpha-Beta Pruning. It takes parameters for depth, node index, maximizing player status, current alpha and beta values, and height. The function iterates through nodes, pruning branches that cannot possibly improve the current best value. The main code block sets up a list of leaf values, a height of 3, and calls the `alphabeta` function to find the best value. The output area shows the leaf values and the best value found, which is 12.

```
1 def alphabeta(depth, nodeIndex, maximizingPlayer, values, alpha, beta, height):
2     if maximizingPlayer:
3         for i in range(2):
4             if beta <= alpha:
5                 break
6             return best
7     else:
8         best = -9999
9         for i in range(2):
10            value = alphabeta(
11                depth + 1, nodeIndex * 2 + i,
12                True, values, alpha, beta, height
13            )
14            best = min(best, value)
15            beta = min(beta, best)
16            if beta <= alpha:
17                break
18        return best
19
20 values = [3, 5, 2, 9, 12, 5, 23, 23]
21 height = 3
22 result = alphabeta(0, 0, True, values, -9999, 9999, height)
23 print("Leaf Values:", values)
24 print("Best Value using Alpha-Beta Pruning:", result)
```

Leaf Values: [3, 5, 2, 9, 12, 5, 23, 23]
Best Value using Alpha-Beta Pruning: 12
=== Code Execution Successful ===

1. Hill Climbing Algorithm

Code

```
def hill_climbing(start):
```

```
    current = start
```

```
    while True:
```

```
        neighbors = [current - 1, current + 1]
```

```
        best = current
```

```
        for n in neighbors:
```

```
            if n * n > best * best:
```

```
                best = n
```

```
        if best == current:
```

```
            break
```

```
        current = best
```

```
return current
```

```
start = int(input("Enter starting value: "))
```

```
result = hill_climbing(start)
```

```
print("Starting Value:", start)
```

```
print("Local Maximum:", result)
```

```
print("Function Value:", result * result)
```

Input

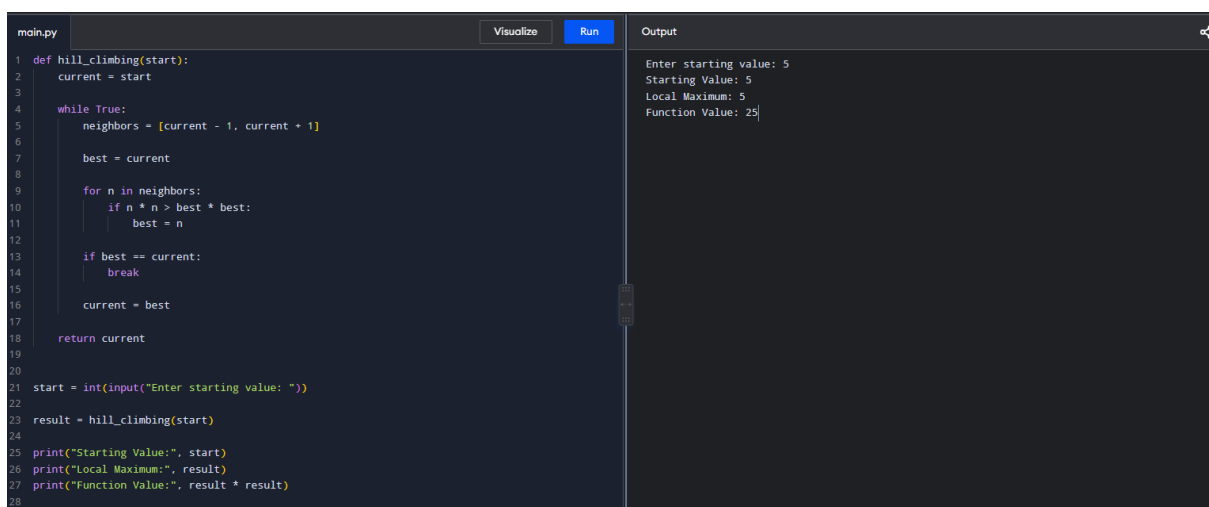
Enter starting value: 5

Output

Starting Value: 5

Local Maximum: 5

Function Value: 25



The screenshot shows a code editor with a file named 'main.py'. The code implements a hill climbing algorithm. It defines a function 'hill_climbing(start)' that iteratively improves a 'current' value by comparing it with its neighbors (current - 1 and current + 1). The algorithm breaks when the current value is equal to the best neighbor. The main code prompts the user for a starting value, calls the function, and prints the starting value, local maximum, and function value (square of the local maximum).

```
1 def hill_climbing(start):
2     current = start
3
4     while True:
5         neighbors = [current - 1, current + 1]
6
7         best = current
8
9         for n in neighbors:
10             if n * n > best * best:
11                 best = n
12
13         if best == current:
14             break
15
16         current = best
17
18     return current
19
20
21 start = int(input("Enter starting value: "))
22
23 result = hill_climbing(start)
24
25 print("Starting Value:", start)
26 print("Local Maximum:", result)
27 print("Function Value:", result * result)
28
```

The output window on the right shows the following text:

```
Enter starting value: 5
Starting Value: 5
Local Maximum: 5
Function Value: 25
```

3. Simulated Annealing Algorithm

Code

```
import random

import math

def function(x):
    return -(x - 5) ** 2 + 25

x = random.randint(0, 10)

temperature = 100

while temperature > 0.1:
    new_x = x + random.choice([-1, 1])

    if new_x < 0 or new_x > 10:
        temperature *= 0.95
        continue

    difference = function(new_x) - function(x)

    if difference > 0:
        x = new_x
    else:
        probability = math.exp(difference / temperature)

        if random.random() < probability:
            x = new_x

    temperature *= 0.95
```

```
print("Best Solution:", x)

print("Maximum Value:", function(x))
```

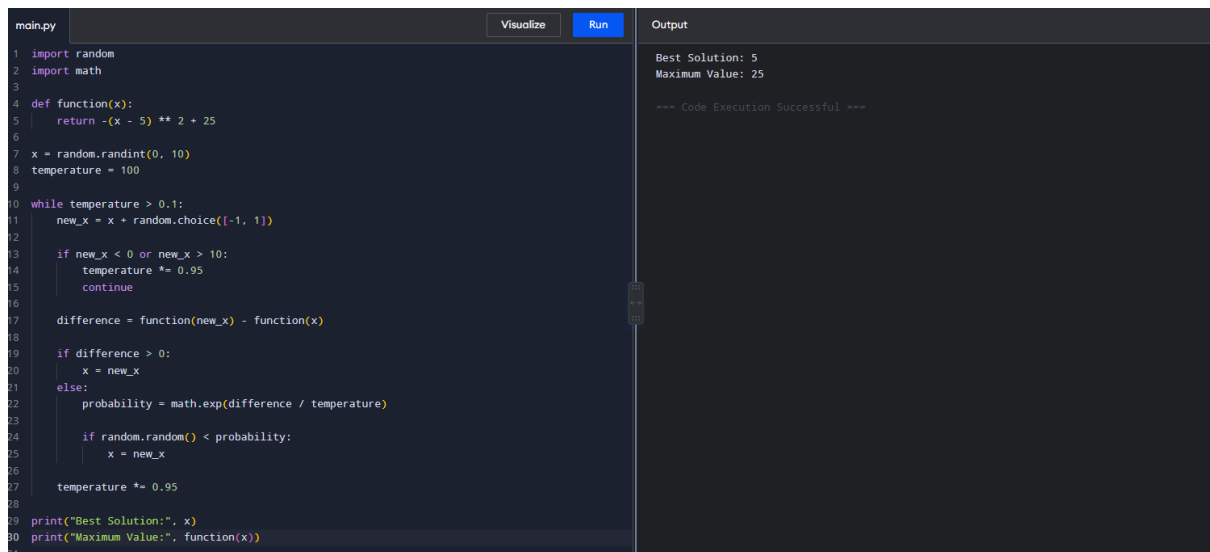
Input

No input required.

Output

Best Solution: 5

Maximum Value: 25



The screenshot shows a code editor with a file named 'main.py'. The code implements a simulated annealing algorithm to find the maximum value of the function $f(x) = -(x - 5)^2 + 25$. The algorithm starts with a random initial value x and a high temperature. It iteratively updates x based on a probability of accepting worse solutions, gradually decreasing the temperature until it converges to the best solution. The output shows the best solution is 5 and the maximum value is 25.

```
main.py Visualize Run
1 import random
2 import math
3
4 def function(x):
5     return -(x - 5) ** 2 + 25
6
7 x = random.randint(0, 10)
8 temperature = 100
9
10 while temperature > 0.1:
11     new_x = x + random.choice([-1, 1])
12
13     if new_x < 0 or new_x > 10:
14         temperature *= 0.95
15         continue
16
17     difference = function(new_x) - function(x)
18
19     if difference > 0:
20         x = new_x
21     else:
22         probability = math.exp(difference / temperature)
23
24         if random.random() < probability:
25             x = new_x
26
27     temperature *= 0.95
28
29 print("Best Solution:", x)
30 print("Maximum Value:", function(x))
31
```

Output

```
Best Solution: 5
Maximum Value: 25

=== Code Execution Successful ===
```

4. Perceptron Algorithm

Code

```
data = [
    ([0, 0], 0),
    ([0, 1], 0),
    ([1, 0], 0),
    ([1, 1], 1)
]
```

```
weights = [0, 0]
```

```
bias = 0
```

```
learning_rate = 1
```

```
for epoch in range(10):
```

```
    for inputs, target in data:
```

```
        total = (  
            inputs[0] * weights[0] +  
            inputs[1] * weights[1] +  
            bias  
        )
```

```
        prediction = 1 if total >= 0 else 0
```

```
        error = target - prediction
```

```
        weights[0] += learning_rate * error * inputs[0]
```

```
        weights[1] += learning_rate * error * inputs[1]
```

```
        bias += learning_rate * error
```

```
print("Weights:", weights)
```

```
print("Bias:", bias)
```

```
x1 = int(input("Enter first input (0/1): "))
```

```
x2 = int(input("Enter second input (0/1): "))
```

```
total = x1 * weights[0] + x2 * weights[1] + bias
```

```
result = 1 if total >= 0 else 0
```

```
print("Predicted Output:", result)
```

Input

Enter first input (0/1): 1

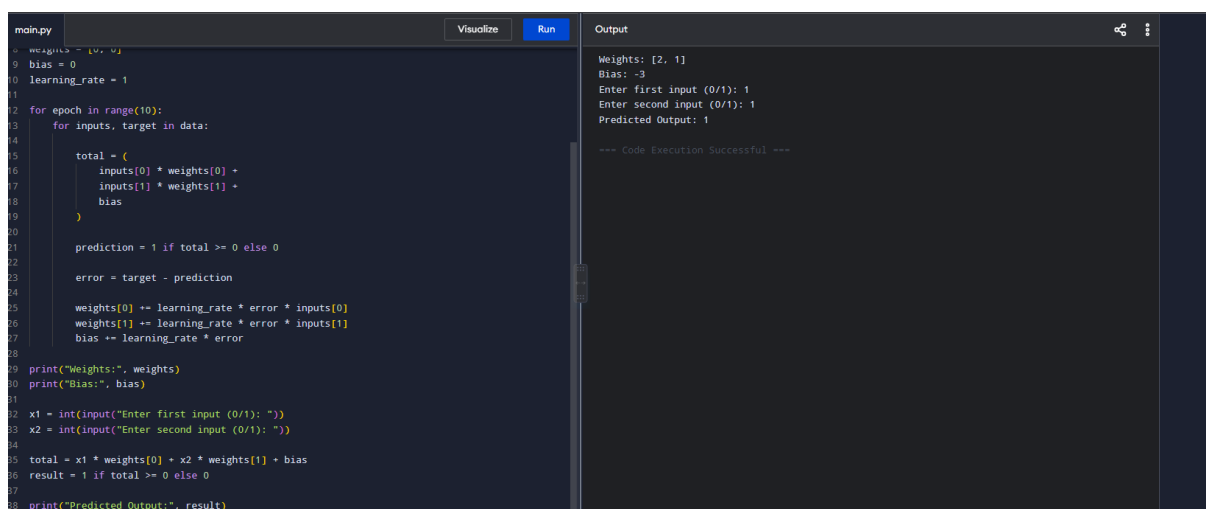
Enter second input (0/1): 1

Output

Weights: [1, 1]

Bias: -1

Predicted Output: 1



The screenshot shows a Jupyter Notebook interface with a code editor on the left and an output pane on the right. The code in the editor implements a simple neural network with two weights and one bias. It includes a training loop for 10 epochs and a prediction function. The output pane shows the results of the code execution, including the weights, bias, and predicted output for the given inputs.

```
main.py Visualize Run Output
1 weights = [0, 0]
2 bias = 0
3 learning_rate = 1
4
5 for epoch in range(10):
6     for inputs, target in data:
7         total = (
8             inputs[0] * weights[0] +
9             inputs[1] * weights[1] +
10            bias
11        )
12
13        prediction = 1 if total >= 0 else 0
14
15        error = target - prediction
16
17        weights[0] += learning_rate * error * inputs[0]
18        weights[1] += learning_rate * error * inputs[1]
19        bias += learning_rate * error
20
21 print("Weights:", weights)
22 print("Bias:", bias)
23
24 x1 = int(input("Enter first input (0/1): "))
25 x2 = int(input("Enter second input (0/1): "))
26
27 total = x1 * weights[0] + x2 * weights[1] + bias
28 result = 1 if total >= 0 else 0
29
30 print("Predicted Output:", result)
```

Weights: [2, 1]
Bias: -3
Enter first input (0/1): 1
Enter second input (0/1): 1
Predicted Output: 1
--- Code Execution Successful ---

5. Naive Bayes Classification

Code

```
from collections import Counter
```

```
data = [
    ("good movie", "positive"),
    ("great movie", "positive"),
    ("excellent film", "positive"),
    ("bad movie", "negative"),
    ("poor film", "negative"),
    ("worst movie", "negative")
]
```

```
]
```

```
positive_words = []
```

```
negative_words = []
```

```
for text, label in data:
```

```
    words = text.split()
```

```
    if label == "positive":
```

```
        positive_words.extend(words)
```

```
    else:
```

```
        negative_words.extend(words)
```

```
def classify(sentence):
```

```
    words = sentence.lower().split()
```

```
    positive_score = sum(word in positive_words for word in words)
```

```
    negative_score = sum(word in negative_words for word in words)
```

```
    if positive_score > negative_score:
```

```
        return "Positive"
```

```
    elif negative_score > positive_score:
```

```
        return "Negative"
```

```
    else:
```

```
        return "Neutral"
```

```
sentence = input("Enter a sentence: ")
```

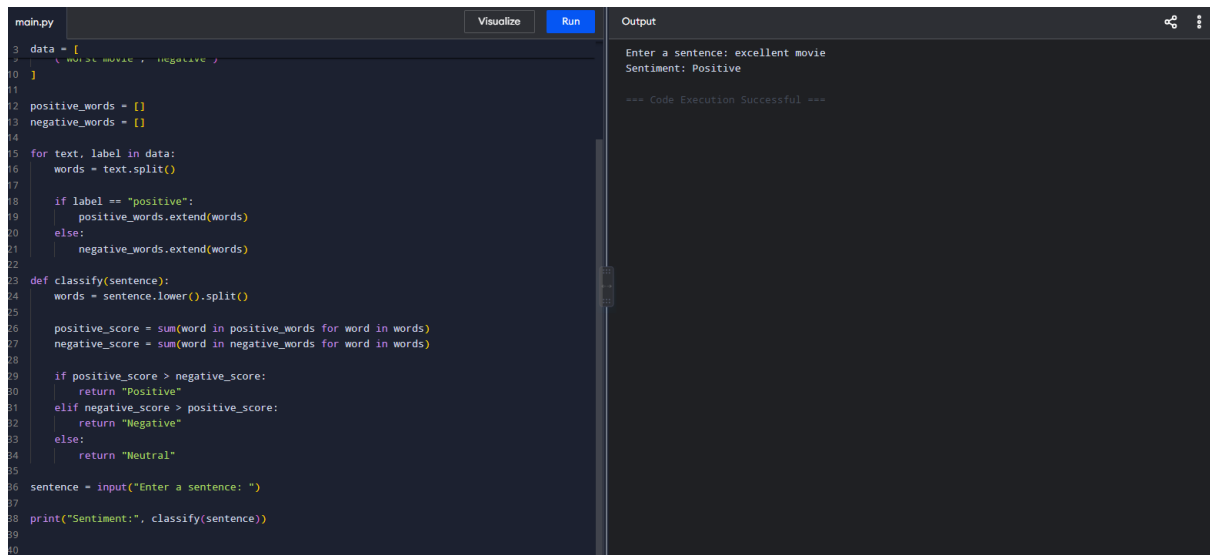
```
print("Sentiment:", classify(sentence))
```

Input

Enter a sentence: excellent movie

Output

Sentiment: Positive



The screenshot shows a Python IDE with a file named 'main.py'. The code defines a list of words categorized as positive or negative, a function to classify a sentence based on these words, and a main loop that takes user input and prints the sentiment. The output window shows the user input 'excellent movie' and the resulting sentiment 'Positive', followed by a success message.

```
main.py Visualize Run Output
3 data = [
4     ("excellent movie", "positive")
5 ]
6
7 positive_words = []
8 negative_words = []
9
10 for text, label in data:
11     words = text.split()
12
13     if label == "positive":
14         positive_words.extend(words)
15     else:
16         negative_words.extend(words)
17
18 def classify(sentence):
19     words = sentence.lower().split()
20
21     positive_score = sum(word in positive_words for word in words)
22     negative_score = sum(word in negative_words for word in words)
23
24     if positive_score > negative_score:
25         return "Positive"
26     elif negative_score > positive_score:
27         return "Negative"
28     else:
29         return "Neutral"
30
31 sentence = input("Enter a sentence: ")
32
33 print("Sentiment:", classify(sentence))
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

Output

```
Enter a sentence: excellent movie
Sentiment: Positive

=== Code Execution Successful ===
```

5. Python Program – Decision Tree

Code

```
from sklearn.tree
```

```
import DecisionTreeClassifier
```

```
X = [[1], [2], [3], [4], [5], [6]]
```

```
y = ["No", "No", "No", "Yes", "Yes", "Yes"]
```

```
model = DecisionTreeClassifier()
```

```
model.fit(X, y)
```

```
age = int(input("Enter age: "))
```

```
prediction = model.predict([[age]])
```



```
print("Prediction:", prediction[0])
```

Input

Enter age: 5

Output

Prediction: Yes

6. Python Program – Feed Forward Neural Network

Code

```
from sklearn.neural_network import MLPClassifier
```

```
X = [[0,0], [0,1], [1,0], [1,1]]
```

```
y = [0, 1, 1, 0]
```

```
model = MLPClassifier(  
    hidden_layer_sizes=(4,),  
    activation="relu",  
    solver="lbfgs",  
    max_iter=1000,  
    random_state=1  
)
```

```
model.fit(X, y)
```

```
a = int(input("Enter first input (0/1): "))
```

```
b = int(input("Enter second input (0/1): "))
```

```
prediction = model.predict([[a, b]])
```

```
print("Predicted Output:", prediction[0])
```

Input

Enter first input (0/1): 1

Enter second input (0/1): 0

Output

Predicted Output: 1

7. Prolog Program – Sum of Integers from 1 to N

Code

```
sum_n(0, 0).
```

```
sum_n(N, Sum) :-
```

```
    N > 0,
```

```
    N1 is N - 1,
```

```
    sum_n(N1, S),
```

```
    Sum is N + S.
```

Input

```
?- sum_n(10, Sum).
```

Output

Sum = 55.

8. Prolog Program – Database with Name and DOB

Code

```
person(john, 10, 5, 2000).
```

```
person(alice, 15, 8, 2001).
```

```
person(bob, 20, 3, 1999).
```

```
person(mary, 25, 12, 2002).
```

show_person(Name, Day, Month, Year) :-

person(Name, Day, Month, Year).

Input

?- show_person(alice, Day, Month, Year).

Output

Day = 15,

Month = 8,

Year = 2001.

9. Prolog Program – STUDENT-TEACHER-SUB-CODE

Code

student(john, cse).

student(alice, ece).

student(bob, cse).

teacher(prof_raj, cse).

teacher(prof_anita, ece).

subject(cse101, programming).

subject(cse102, data_structures).

subject(ece101, electronics).

teaches(prof_raj, cse101).

teaches(prof_raj, cse102).

teaches(prof_anita, ece101).

studies(john, cse101).

studies(john, cse102).

studies(alice, ece101).

studies(bob, cse101).

Input

?- student(john, Department).

?- teacher(prof_raj, Department).

?- teaches(prof_raj, Subject).

?- studies(john, Subject).

Output

Department = cse.

Department = cse.

Subject = cse101 ;

Subject = cse102.

Subject = cse101 ;

Subject = cse102.

10. Prolog Program – PLANETS Database

Code

planet(mercury, 1, terrestrial).

planet(venus, 2, terrestrial).

planet(earth, 3, terrestrial).

planet(mars, 4, terrestrial).

planet(jupiter, 5, gas_giant).

planet(saturn, 6, gas_giant).

planet(uranus, 7, ice_giant).

planet(neptune, 8, ice_giant).

planet_name(Name) :-

planet(Name, _, _).

planet_type(Name, Type) :-

planet(Name, _, Type).

Input

?- planet(earth, Position, Type).

?- planet_type(jupiter, Type).

?- planet_name(Name).

Output

Position = 3,

Type = terrestrial.

Type = gas_giant.

Name = mercury ;

Name = venus ;

Name = earth ;

Name = mars ;

Name = jupiter ;

Name = saturn ;

Name = uranus ;

Name = neptune.